

NodeRock: An Approach for Selecting Tests Prone to Event Races in Node.js Applications

Pedro Vinícius Guandalini Vicente
Universidade Federal de São Carlos
São Carlos, São Paulo, Brazil
pedrovicente@estudante.ufscar.br

Vinicius H. S. Durelli
Universidade Federal de São Carlos
São Carlos, São Paulo, Brazil
vinicius.durelli@ufscar.br

Rogério Galante Negri
Universidade Estadual Paulista
São José dos Campos, São Paulo, Brazil
rogerio.negri@unesp.br

André Takeshi Endo
Universidade Federal de São Carlos
São Carlos, São Paulo, Brazil
andreendo@ufscar.br

Abstract

Context: Node.js is widely used for developing JavaScript applications, employing an efficient asynchronous execution model. However, the nondeterminism of this model can lead to event races, which are subtle nondeterministic behaviors that may trigger concurrency bugs and compromise software reliability. Current detection approaches often require exhaustive analysis of entire test suites, resulting in high computational costs and lacking mechanisms to select and prioritize tests prone to event races.

Goal: This paper proposes and evaluates NodeRock, a dynamic analysis approach designed to select and prioritize test cases prone to event races in Node.js applications.

Method: NodeRock instruments test executions to collect traces, extracts 15 dynamic features characterizing asynchronous behavior, and employs machine learning models to classify tests based on their event race proneness.

Results: The results of NodeRock’s evaluation on 24 open-source Node.js projects are promising, suggesting that dynamic execution metrics provide a viable strategy for test selection and prioritization.

Conclusion: NodeRock shows that dynamic execution metrics can serve as indicators of event race proneness in tests, offering a practical way to optimize existing detection tools.

Keywords

JavaScript, Software Testing, Event Races, Machine Learning, Automated Testing, Test Case Prioritization, Test Suite Reduction

1 Introduction

JavaScript is a dynamic, multi-paradigm programming language that has consolidated its position as one of the most widely used languages among developers [22, 37, 41]. With the introduction of the Node.js runtime environment¹, JavaScript expanded beyond the browser into server-side applications, microservices, desktop tools, and command-line interfaces. Node.js features a single-threaded, non-blocking architecture that achieves efficiency and scalability by delegating computationally expensive tasks and I/O operations to background worker threads [40]. Its ecosystem is further reinforced by npm², which provides over two million open-source packages used by global organizations such as NASA, Walmart, and Uber [11].

However, the unpredictable completion order of asynchronous operations introduces a class of concurrency bugs specific to event-driven environments. Because worker threads operate independently and their completion order is influenced by external factors such as system load and I/O latency, callbacks and event handlers may be scheduled in an order not anticipated by the developer [20, 21, 45]. This phenomenon gives rise to *event races*, which are race conditions generated by nondeterministic orderings in the execution of asynchronous events. Their consequences range from application crashes and data inconsistency to security vulnerabilities and *flaky tests* (i.e., tests that pass and fail nondeterministically [27]).

To address event races, the literature has proposed two main detection techniques. The first relies on *happens-before* modeling to identify causal relationships between events, as exemplified by NodeRacer [21], NRace [9], and NodeRT [45]. The second employs delay injection to probabilistically expose races through repeated re-executions, as in the Node.js Asynchronous Callback Delayer (NACD) [20]. While effective, both techniques may impose a high computational cost, either from constructing complex execution models or from the need for extensive re-executions. This cost makes exhaustive analysis of modern test suites with hundreds or thousands of test cases impractical, highlighting the lack of a mechanism in the literature for selecting and prioritizing tests prone to event races before invoking expensive detection tools.

The software testing community has established two principal strategies for managing test suite costs, namely Test Case Prioritization (TCP) [34] and Test Suite Reduction (TSR) [24]. TCP reorders tests to maximize early fault detection, while TSR selects a representative subset to reduce overall execution cost [43]. However, both strategies often rely on heuristics grounded in branch and code coverage, historical failure data, or code change analysis [18, 31, 36], which may not fully capture the nondeterministic and intermittent nature of event races.

In this context, we hypothesize that the dynamic behavior of a test in terms of callback usage, Promise³ lifecycles, and event scheduling patterns can serve as an indicator of tests prone to event races, supporting TCP and TSR strategies for asynchronous software. Rather than trying to directly detect an event race, our approach extracts a feature vector from instrumented test executions

¹<https://nodejs.org/>

²<https://www.npmjs.com/>

³A Promise is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value [17, 29].

and applies machine learning (ML) [31] to classify tests based on their proneness to event races. By incorporating this classification, developers can reduce the test suite to a subset of tests prone to event races (TSR) and prioritize the execution of these tests (TCP), allowing them to focus debugging efforts on a prioritized subset of tests, making event race detection more practical at scale.

To realize this vision, we propose and evaluate NodeRock, a dynamic analysis approach that instruments Node.js test executions to extract features characterizing asynchronous behavior and applies a Positive-unlabeled (PU) classifier⁴[19] to select tests prone to event races. To capture these characteristics, NodeRock dynamically instruments each test execution to collect 15 carefully proposed features, including metrics such as function call counts, execution durations, asynchronous context, and Promise-related features, such as resolution and rejection times. These features represent the dimensions of asynchronous execution that we hypothesize to correlate with event race proneness.

We evaluate NodeRock on 24 open-source projects comprising 1,660 automated tests and 32 confirmed event races. Regarding the effectiveness of ML models, the results are promising: the PU classifier achieved the best trade-off, with 84.38 % recall and 75 % accuracy. Furthermore, Information Gain analysis indicates that the collected metrics carry meaningful predictive value, with callback-related features and long-interval asynchronous operations emerging as the strongest indicators. Regarding runtime overhead, a study on a single project (node-archiver) showed that NodeRock reduced total execution time by 6.02 % in an intensive re-execution scenario while preserving detection capability.

This paper is structured as follows. Section 2 presents a motivating example illustrating how tests can be prone to event races. Section 3 describes the NodeRock approach, covering its pipeline, dataset and implementation. Section 4 reports the empirical evaluation. Section 5 addresses the threats to validity. Section 6 discusses related works and situates NodeRock in the literature. Finally, Section 7 presents conclusions and directions for future works.

2 Motivating Example

To demonstrate in practice how event races can lead to intermittent and hard-to-debug failures in Node.js applications, we analyze a real-world event race detected by the NACD tool [20] in the `fs-extra` library⁵, specifically in its tests associated with the file copying feature.

The core issue originates from the `readDirFiles` function, whose simplified implementation is shown in Listing 1. This function is designed to recursively read the contents of a directory asynchronously. It iterates over directory entries using `async.forEach` (Line 4). For each entry, it triggers asynchronous operations such as `fs.stat` (Line 5) to check its type. If it is a directory (Line 6), a recursive call to `readDirFiles` is made (Line 7). If it is a file, `fs.readFile` is called to read the file's contents (Line 12). In both cases, the shared object `result` (instantiated in Line 2) is populated from the respective callbacks of these asynchronous operations (Lines 8 and 13). And, at the end of `forEach`, the function passed as

callback to `readDirFiles` is executed over the found file structure (Line 18).

```

1  function readDirFiles(dirPath, callback) {
2    var result = {};
3    fs.readdir(dirPath, (files) => {
4      async.forEach(files, (file, next) => {
5        fs.stat(file, (stat) => {
6          if (stat.isDirectory()) {
7            readDirFiles(file, (subFiles) => {
8              result[file] = subFiles;
9              next();
10             });
11           } else {
12             fs.readFile(file, (content) => {
13               result[file] = content;
14               next();
15             });
16           }
17         });
18       }, () => callback(result));
19     }
20   }

```

Listing 1:
Representation of `readDirFiles`, showing the asynchronous population of the result object that leads to an event race.
Adapted from: <https://github.dev/mmalecki/node-read-dir-files#readdirfilesreaddir-encoding-recursive-callback>.

Since `async.forEach` processes entries concurrently and the completion of these I/O operations is affected by external factors (like disk access time and workload of worker threads), the execution flow does not follow a deterministic order. Thus, the order in which files or subdirectories are added to the `result` object may vary between runs. For example, in one execution, the “sub” subdirectory may be processed and added before the “a” file, while in another run, the opposite may occur. This variation in data order, caused by the nondeterministic behavior of asynchronous calls, characterizes an event race. This behavior is illustrated in Figure 1, which depicts the analyzed instability.

This nondeterminism in the order of properties in the object returned by `readDirFiles` becomes problematic when consumed by code that depends on that order. An example is the `fs-extra` test illustrated in Listing 2. The test applies a `filter` function (Lines 24–32, called in Line 33) to the `srcFiles` object (obtained as result from `readDirFiles`) to remove files whose names end with “a”. However, the original implementation of `filter` contained a logical flaw: on Lines 26 and 27, upon encountering a subdirectory (`curFile instanceof Object`), the function made a recursive call to explore the files within that subdirectory, but immediately returned (`return filter(curFile)`), without processing the remaining items at the current level of the `curFile`.

The combination of the event race in `readDirFiles` and the logical flaw in `filter` resulted in a flaky test. If, due to the event race, `readDirFiles` constructed the `srcFiles` object in a way that a subdirectory (“sub”) appeared before the file “a”

⁴A PU classifier is designed for scenarios where only positive examples are reliably labeled. It treats the unlabeled data as a mixture of positive and negative samples to learn decision boundaries [19].

⁵<https://github.com/jprichardson/node-fs-extra>

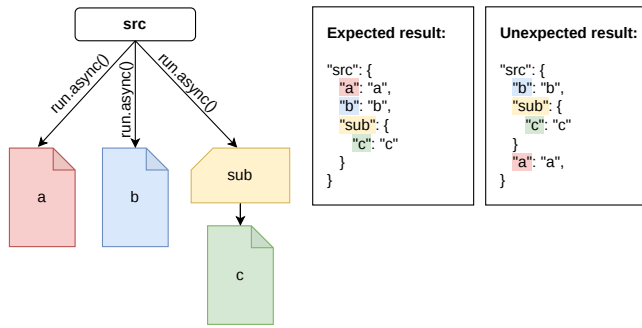


Figure 1: Illustration of an event race caused by asynchronous processing in readDirFiles. The asynchronous handling can lead to nondeterministic insertion order in the “result” object. The expected order (“a”, “b”, “sub”) may vary across executions, as shown in the unexpected result where “sub” appears before “a”.

```

21 it('filters out files ending with "a"', () => {
22   const src = path.join(__dirname, 'analyzedDirectory');
23   readDirFiles(src, (srcFiles) => {
24     function filter(files) {
25       for (const fileName in files) {
26         const curFile = files[fileName];
27         if (curFile instanceof Object) // curFile is a
            subdirectory
28           return filter(curFile); // DFS
29         if (fileName.endsWith("a"))
30           delete files[fileName];
31       }
32     }
33     filter(srcFiles);
34
35     expectedSrcFiles = {
36       "src": {
37         "b": "b",
38         "sub": {
39           "c": "c"
40         }
41       }
42     }
43     assert.deepStrictEqual(srcFiles, expectedSrcFiles);
44   });
45 });
  
```

Listing 2: Representation of the test and its use of filter function responsible for the flaky behavior. The flawed early return causes some files to be skipped depending on the order induced by the event race. Adapted from: <https://github.com/jprichardson/node-fs-extra/pull/737/files>

at the same level, the filter function, upon processing “sub”, would return prematurely, never evaluating and thus never removing the file “a”. This would lead to a failure in the assertion `assert.deepStrictEqual(srcFiles, expectedSrcFiles)`

(Line 43), since `srcFiles` would still contain file “a”. However, if file “a” were processed before the subdirectory, it would be correctly removed, and the test would pass. This inconsistency in test results, dependent on the order of asynchronous events, is a characteristic of a flaky test induced by an event race [15, 20, 21].

The solution to this unstable behavior, proposed in the pull request⁶ that fixed the flaky test, was to remove the return statement from the recursive call in the filter function (Line 28). This change alters the function’s logic, instead of a Depth-First Search that explores the first found subdirectory and terminates the iteration at the current level, it allows the for loop to continue processing the remaining files and subdirectories at the same level, even after the recursive call completes.

This case exemplifies the practical cost of existing detection approaches. The race was identified by NACD [20] only after repeatedly re-executing the test suite with injected delays to perturb event scheduling. Applying this strategy exhaustively across an entire test suite, which may contain hundreds or even thousands of tests (e.g., 719 tests in `fs-extra`), of which the vast majority are race-free, represents a substantial computational expense. To optimize the testing process, a possible approach is to identify tests prone to event races before invoking computationally expensive detection tools. The observable execution profile of the flaky test in this example, such as its intensive use of callback functions, is the type of signal that NodeRock aims to capture and use to direct the detection effort towards tests prone to event races.

3 Approach

NodeRock is a dynamic analysis approach designed to select and prioritize tests prone to event races in Node.js applications. Rather than detecting bugs, it acts as a triage layer by analyzing the asynchronous execution trace of each test and identifying those whose runtime behavior resembles that of tests with confirmed event races, thereby allowing developers to direct detection tools, such as NodeRacer [21], NRace [9], NodeRT [45] and NACD [20], only to tests for which they are most likely to be needed.

3.1 The NodeRock Pipeline

NodeRock is implemented as a command-line interface (CLI) tool that interacts with a target project via its file system path, without the need for direct changes to the analyzed project source code. Once invoked, the tool automatically proceeds through three sequential phases, as illustrated in Figure 2, namely (i) Test Collection, (ii) Feature Extraction, and (iii) ML Selection.

Test Collection Phase: The first phase aims to identify the candidate tests in the automated test suite of the target Node.js project. NodeRock integrates with Mocha⁷ and Jest⁸, which are two widely-used test automation frameworks in the Node.js ecosystem. This integration allows the tool to automatically collect the names and file paths of all currently passing test cases, ensuring that only valid tests, which might contain event races, are considered for the subsequent dynamic analysis phases.

⁶<https://github.com/jprichardson/node-fs-extra/pull/737/files>

⁷<https://mochajs.org/>

⁸<https://jestjs.io/>

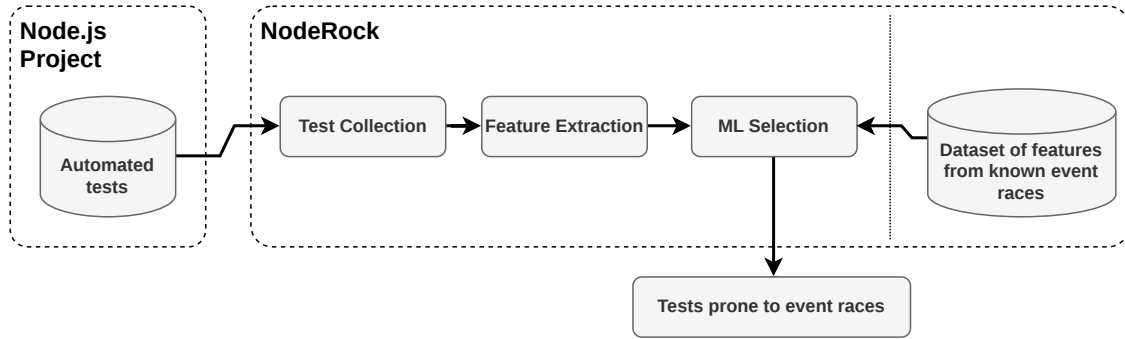


Figure 2: Representation of NodeRock execution phases during an analysis.

Feature Extraction Phase: For each candidate test, NodeRock extracts 15 features, in the form of dynamic metrics, that are intended to characterize its asynchronous behavior. The extraction process is divided into two sequential, isolated runs to mitigate interference. The first run instruments the test to capture an execution trace that includes function call counts, execution durations, and asynchronous context identifiers. The second run collects Promise-related features by intercepting creation and settlement events, recording lifecycle details such as resolution and rejection times. Although event scheduling is non-deterministic, the extracted metrics reflect overall asynchronous workload rather than precise interleaving sequences, thus, combining the two feature sets does not require strictly identical scheduling between runs.

The extracted features from both runs are combined into a single feature vector per test. These features are then normalized using per-project MinMax scaling⁹ to prevent disparities in project magnitudes from biasing the results in the ML Selection phase. Consequently, each test is represented by a 30-dimensional vector that concatenates the 15 raw metrics with their respective per-project normalized variants, allowing the ML model to consider both absolute values and relative distributions.

Table 1 presents the complete feature set, descriptions and rationale, organized into two groups that correspond to the two collection runs described above. The first group, *Execution Trace Analysis (via Hooks)*, contains ten metrics derived from the instrumented execution trace captured using NodeProf [39] hooks, which are used by NodeRT [45] and extended by NodeRock. These metrics are organized into five subgroups according to the behavior they capture. Metrics #1 and #2 quantify general function invocation activity: the total number of invoked functions and the count of long-running invocations reflect overall execution complexity and the temporal windows in which concurrent events may intercalate. Metrics #3 to #5 focus on callback usage, which is one of the core mechanisms of asynchronous behavior in Node.js, as illustrated in Section 2, where a high number of callbacks was used in a test containing the event race. Metrics #6 to #8 capture the use of `async/await` syntax, a modern counterpart to callback-based programming, whose presence signals asynchronous operations

whose completion order may vary across executions. Metric #9 is named `Unique_Asynchhook_IDs`, which counts the number of simultaneously active asynchronous contexts; a higher value indicates greater potential for unexpected event orderings. Metric #10 contains `Tests_Total_Duration`, which captures the overall test execution time, as longer tests provide a wider temporal window for races and flaky tests to manifest [2].

The second group, *Analysis of Promises (via Monkey Patching)*, contains metrics #11 to #15 that characterize the Promise lifecycle by capturing the volume and timing of settlement events, including resolution and rejection patterns that were not accessible through instrumentation hooks. For delay-related metrics (#2 and #4), a heuristic threshold of 100 ms was arbitrarily adopted as a practical boundary to filter out trivial operations and focus on temporal windows where nondeterministic concurrent scheduling is more likely to produce event races, though we acknowledge this choice was not empirically optimized.

ML Selection Phase: The final phase classifies tests as prone to event races or not. In our analysis, this problem does not conform to standard binary classification. While tests with confirmed event races constitute a positive class, the remaining tests are not confirmed negatives, they are unlabeled. A test with no known race may simply be one whose race has not yet been discovered. Treating unlabeled tests as negatives could distort the decision boundary.

We therefore frame the task as a PU classification problem [19], which is designed for settings where only the positive class is reliably known. The PU classification algorithm treats unlabeled data as a mixture of positive and negative samples, using a Random Forest [3] as the base estimator to learn decision boundaries. The empirical justification for this choice over supervised alternatives is reinforced by the results in Section 4.1. Once trained, the model assigns each investigated test a probability score and tests above a confidence threshold are flagged as prone, producing a prioritized list for developer investigation. The dataset used to train this classifier, its experimental setup, and the corresponding results are detailed in Sections 3.2, 4, and 4.1, respectively.

3.2 Dataset Selection

To train and validate the ML models, we aggregated open-source Node.js projects from prior event race detection studies [20, 21, 45].

⁹<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>

Table 1: Metrics extracted during feature extraction and Promises monkey patching phases.

ID	Metric Name	Description	Rationale
Execution Trace Analysis (via Hooks)			
#1	Invoked_Functions	Number of functions invoked during execution.	A higher number of invoked and long-running functions indicates greater execution complexity. Callbacks are one of the primary mechanisms through which asynchronous results are delivered.
#2	Invokes_Interval_Greater_100ms	Number of function invocations whose execution duration exceeds 100 milliseconds.	
#3	Invoked_Callbacks	Number of function invocations involving the use of callbacks.	
#4	Cb_Delays_Greater_100ms	Number of callbacks whose wait time (delay) to start execution exceeds 100 milliseconds.	
#5	Cb_Total_Delay	Sum, in milliseconds, of the wait times (delay) for all callbacks.	async/await syntax is the contemporary alternative to callback-based asynchronous programming.
#6	Async_Function_Count	Number of functions declared with the async keyword.	
#7	Await_Count	Number of await expressions used in the code.	
#8	Await_Intervals	Time interval, in milliseconds, between the call of an await expression and the moment execution resumes.	
#9	Unique_Asynchook_IDs	Number of unique async_hooks identifiers used to track asynchronous operations.	A higher number of unique asynchronous context identifiers indicates more active asynchronous operations.
#10	Tests_Total_Duration	Total duration of the test execution, expressed in seconds.	Longer test executions provide a wider temporal window for event races and flaky tests to manifest [2].
Analysis of Promises (via Monkey Patching)			
#11	Total_Settled_Promises	Number of Promises that were completed (settled), either successfully (resolved) or with an error (rejected).	Promises are also a core abstraction for asynchronous operations in modern JavaScript.
#12	Avg_Resolved	Average resolution time, in milliseconds, for successful Promises.	
#13	Avg_Rejected	Average rejection time, in milliseconds, for rejected Promises.	
#14	Longest_Resolved	Time, in milliseconds, of the slowest Promise to be successfully resolved.	
#15	Resolved_Percentage	Percentage of resolved Promises relative to the total number of settled Promises.	

Projects that could not be successfully built in our environment or were incompatible with the NodeRock instrumentation were excluded, as feature extraction would not be possible for such projects.

The resulting dataset is summarized in Table 2. Each row corresponds to one project and reports three pieces of information. The Project column identifies the project by name. The #Tests column reports the number of passing tests that are eligible for feature extraction by NodeRock. Projects marked with “-” were contributed by the originating studies as isolated test scripts targeting a specific race, rather than as complete project suites, so their full test count is not available. These projects contribute their race-exposing script as a positive instance and add no other instances to the dataset, which slightly enriches the positive class with tests written specifically to expose a race. The last column reports the number of tests with confirmed event races as identified and reported by the originating studies. These constitute the positive class used to train and evaluate the ML models. The final dataset comprises 24 projects, 1,660 automated tests, and 32 confirmed event races.

3.3 Implementation

NodeRock is built upon NodeRT [45], which in turn relies on NodeProf [39] for dynamic analysis of JavaScript on the GraalVM virtual machine. This architecture, implemented in JavaScript and TypeScript using around 10,300 lines of code, enables NodeRock to intercept execution events without modifying the application

source code or Node.js internals. Due to dependencies on specific versions of this instrumentation stack, the current implementation is compatible with Node.js 14.

The first instrumentation run extends the hook infrastructure provided by NodeRT [45], registering information for hooks such as functionEnter, functionExit, invokeFunPre, asyncFunctionEnter, awaitPre, and awaitPost, exposed by NodeProf [39]. From these hooks, NodeRock derives nine execution trace metrics, including Invoked_Functions, Invoked_Callbacks, Async_Function_Count, and Await_Count, among others. Additionally, each hook event is annotated with the active asynchronous context identifier retrieved via Node.js’s async_hooks module¹⁰, from which the Unique_Asynchook_IDs metric is derived. The second run applies monkey patching¹¹ to the global Promise object by replacing it with an instrumented subclass that intercepts each Promise constructor call and proxies its resolution and rejection callbacks to record timing information. From this instrumentation, five Promise lifecycle metrics are derived: Total_Settled_Promises, Avg_Resolved, Avg_Rejected, Longest_Resolved, and Resolved_Percentage. Isolating these two techniques into separate executions prevents the overhead of one run from interfering with the timing metrics of the other.

¹⁰https://nodejs.org/api/async_hooks.html

¹¹Monkey patching is a technique used to modify the behavior of software components at runtime without altering the original source code [5].

Table 2: Project Dataset Summary.

Project	#Tests	#Tests w/ Ev. Races
Mongo-express	33	4
Nedb	330	2
Node-archiver	35	1
Objection.js	48	1
Agent Keep Alive	24	1
fiware-pep-steelskin	210	1
WhiteboxGhost	-	1
node-mkdirp	-	1
node-logger-file-1	10	1
socket.io-1862	26	1
del	10	1
linter-stylint	-	1
Node-simpleCrawler	81	1
xlsx-extract	14	1
bluebird-2	-	1
nodesamples (express-user)	-	1
get-port	-	1
live-server-potential-race	-	1
socket.io-client	59	1
json-file-store	27	1
json-fs-store	7	1
ncp	15	4
write	12	1
fs-extra	719	2
Total	1,660	32

Data processing and ML classification are implemented in Python using pandas [28] for data manipulation and `scikit-learn` [32] for feature normalization and model training. The complete source code is publicly available in the Artifact Availability Section.

4 Evaluation

To guide the development and evaluation of NodeRock, we formulate three research questions (RQs):

- **RQ1:** How effective are supervised and semi-supervised ML models in selecting tests prone to event races?
- **RQ2:** What is the discriminative power of individual dynamic features in identifying tests prone to event races?
- **RQ3:** What is the runtime overhead of NodeRock in a practical testing scenario?

This section presents the evaluation of NodeRock across the three research questions formulated in Section 4. RQ1 assesses whether ML classifiers can effectively identify tests prone to event races from dynamic execution features, with particular attention to recall as the primary metric of interest. This focus is justified because high recall minimizes false negatives, ensuring that tests containing actual event races are not overlooked, which would allow event races to remain undetected. RQ2 examines which individual features carry the strongest predictive signal. RQ3 measures the practical cost-benefit of deploying NodeRock as a pre-filter for NACD in a re-execution scenario, quantifying the trade-off between instrumentation overhead and the computational savings achieved. Together, these three questions evaluate both the effectiveness and the viability of the approach.

4.1 RQ1: Effectiveness of ML Models

To answer RQ1, we applied NodeRock to the benchmark dataset of projects containing known event races (detailed in Subsection 3.2). After extracting dynamic features for each test, we trained and evaluated a diverse set of classifiers. Since our evaluation is constrained to the 32 confirmed event races available in the dataset, these findings should be interpreted with caution, as this limited sample size restricts broader generalizations.

Among supervised models, we evaluated Support Vector Machine (SVM) [12], K-Nearest Neighbors (KNN) [13], Naive Bayes [16], Random Forest [3], Decision Tree [4] and a voting Ensemble that classifies a test as positive if any model does so. In all supervised experiments, tests with confirmed event races were assigned positive labels and all remaining tests were treated as negative.

To handle the inherent label uncertainty, we also adapted two clustering algorithms for the labeling task. HDBSCAN [8] and K-Means [26] were applied to labeled and unlabeled data, any test belonging to a cluster with at least three known event races was classified as positive. Additionally, two anomaly detection techniques were adapted for this context. Isolation Forest [25] and One-Class SVM [35] were trained exclusively on tests that did not contain confirmed event races. Consequently, any test displaying distinct metric patterns was classified as an outlier, signaling a potential event race. For Label Spreading [44], we seeded confirmed races as positives and additionally labeled as negative any test with both `Invoked_Callbacks` (Raw) and `Async_Function_Count` (Raw) equal to zero, as these tests exhibit no meaningful asynchronous behavior that could produce races. Finally, the PU approach [19] was applied, with confirmed races treated as the positive class and all remaining tests as unlabeled.

To mitigate the class imbalance present in the original dataset (1:53 ratio), we applied random undersampling prior to model training and evaluation, sampling three negative instances per positive. All models were evaluated using Stratified 10-fold cross-validation, measuring Accuracy, Precision, Recall, and F1-Score, as summarized in Tables 3 and 4. The four metrics carry distinct implications in this context. Recall is the proportion of confirmed races in the selected subset. A false negative removes a real race, which would then escape detection. Precision is the proportion of selected tests that contain a race. A false positive does not compromise detection, as it only adds re-execution cost. Accuracy measures the overall proportion of correct classifications across both classes, but it is dominated by the negative class (75 % of the balanced sample) and can stay high even when a model misses a high number of races, while F1-Score assumes the cost of both error types to be equal. Given this asymmetry, we adopt recall as a primary selection criterion, using precision and accuracy as secondary criteria to keep the selected subset smaller than the original suite.

In Table 3, supervised models achieved high accuracy, ranging from around 76 % to 85 %, with Random Forest and SVM reaching the highest values (85.16 % and 85.94 %) alongside good precision (76 % and 85.00 %, respectively). However, the supervised models struggled significantly with recall, the most critical metric for event race detection. A low recall implies tests with actual event races

Table 3: Performance comparison of supervised algorithms in detecting known event races, where $\geq 80\%$ $70\%–79\%$ $50\%–69\%$ $< 50\%$.

	Random Forest	KNN (K=5)	SVM	Naive Bayes	Decision Tree	Ensemble
Accuracy (%)	85.16	84.38	85.94	76.56	81.25	83.60
Precision (%)	76.00	80.00	85.00	62.50	65.38	64.86
Recall (%)	59.38	50.00	53.13	15.62	53.13	75.00
F1-Score (%)	66.67	61.54	65.38	25.00	58.62	69.57

Table 4: Performance comparison of semi-supervised and clustering-based algorithms, where $\geq 80\%$ $70\%–79\%$ $50\%–69\%$ $< 50\%$.

	HDBSCAN	K-Means	Label Spreading	Isolation Forest	One-Class SVM	PU Classifier
Accuracy (%)	78.13	34.37	34.37	69.53	71.88	75.00
Precision (%)	54.35	26.36	23.47	43.64	46.55	50.00
Recall (%)	78.13	90.63	71.88	75.00	84.38	84.38
F1-Score (%)	64.10	40.84	35.38	55.17	60.00	62.79

would be missed. The best supervised model (Ensemble) reached only 75 % recall, while Naive Bayes fell below 16 %.

Among semi-supervised approaches (Table 4), K-Means achieved the highest recall (90.63 %), but this came at the cost of a low precision of 26.36 % and 34.37 % accuracy, rendering the filter unhelpful by flagging most tests as positive. HDBSCAN achieved 78.13 % accuracy and 54.35 % precision, but its recall (78.13 %) remained lower than the PU classifier’s. The PU classifier emerged as the most balanced strategy, it achieved 84.38 % recall (tied with One-Class SVM) while outperforming both K-Means and One-Class SVM in accuracy (75.00 %) and precision (50.00 %). While HDBSCAN showed slightly higher accuracy, its lower recall (78.13 %) made it less suitable for event race detection in our context. Among the models with the highest recalls, K-Means is set aside due to its low precision. Meanwhile, One-Class SVM, though tied with the PU classifier in recall (84.38 %), is outperformed by it in both accuracy (71.88 % vs. 75.00 %) and precision (46.55 % vs. 50.00 %). Furthermore, the PU classifier’s assumptions also match the nature of our dataset, since it does not require treating unlabeled tests, which may simply contain races not yet discovered, as confirmed negatives, and this combination of empirical and conceptual fit supports its choice as NodeRock’s default model.

Answer to RQ1: The results suggest that ML models can assist in selecting tests prone to event races, but require careful algorithm choice. The semi-supervised PU classifier offered the best trade-off in our results, providing high recall ($> 84\%$) to detect most tests prone to event races while maintaining sufficient precision (50 %) to reduce inspection effort.

4.2 RQ2: Discriminative Power of Individual Features

To answer RQ2, we investigated whether the extracted dynamic features carry meaningful individual discriminative power to identify tests prone to event races. To do so, we performed an Information Gain (IG) analysis on the dataset. To establish a baseline, we introduced three synthetic random control variables (Random_Data_Column_1, Random_Data_Column_2, and Random_Data_Column_3) whose values were drawn from a random distribution. This procedure is inspired by the use of a random probe feature to evaluate attribute relevance and establish a statistical baseline [23, 38]. Consequently, any feature with an IG score comparable to these random controls is assumed to offer no meaningful contribution to the classifier.

The results in Figure 3 show the relevance of our features. For visual clarity, the Figure highlights the 15 metrics that demonstrated the most discriminative power. The most informative metric is Invoked_Callbacks (Normalized) (IG = 0.189). This result is intuitive, as the presence of several callbacks implies more asynchronous interactions in the Event Loop¹², increasing the number of possible event orderings and, consequently, the probability that a harmful ordering will happen. Metrics capturing operation duration, such as Invokes_Interval_Greater_100ms (IG = 0.155) and Cb_Delays_Greater_100ms (Normalized) (IG = 0.103), were also highly relevant, as long-running asynchronous operations or delays can extend the time window for concurrent event interaction.

Not all metrics contribute equally. Avg_Rejected (Normalized) (IG = 0.000081) and Await_Intervals (Raw) (IG = 0.000791) show near-zero discriminative power. This outcome may be attributable to the composition of the dataset since the benchmark projects predominantly use callback-based asynchronous patterns, and modern

¹²The Event Loop is the mechanism responsible for scheduling the execution of asynchronous operations in Node.js [30].

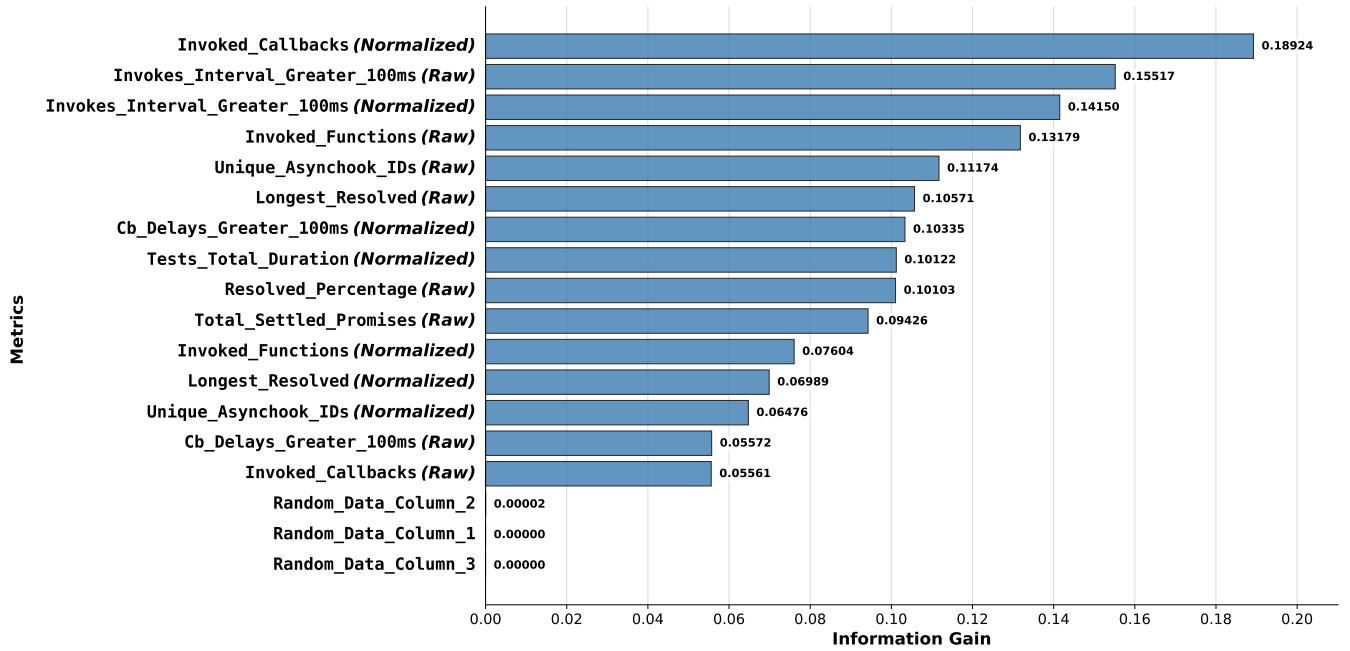


Figure 3: Information Gain values for NodeRock metrics compared to random baselines.

Promise constructs, including `async/await` and rejected Promises, are underrepresented in the dataset. Consequently, these metrics lack sufficient variance in the current dataset to carry a useful signal, though their relevance may increase if the dataset is expanded with more contemporary projects. Critically, all three random control variables achieved $IG \approx 0.000$, indicating that the discriminative capacity observed in the core metrics is not attributable to chance.

Answer to RQ2: The Information Gain analysis provides evidence supporting the usefulness of the feature set. NodeRock’s metrics capture distinct characteristics of asynchronous behavior that correlate with event races, with the features outperforming random noise.

4.3 RQ3: Practical Runtime Performance

Regarding RQ3, we evaluated the practical viability of NodeRock by measuring runtime performance and overhead. Following established practice in event race detection [15, 20, 21], each test was subjected to 100 executions to measure the event race reproduction rate. This process was repeated 15 times for statistical consistency since the used detection tool makes random scheduling decisions [20].

To make it actionable, the analysis was conducted with a single project, `node-archiver`, as it satisfies the criteria of containing a confirmed event race and providing a considerable automated test suite (i.e., 35 tests). The cost-benefit results reported below are therefore specific to this project. The experimental design compared two strategies for detecting event races using the NACD tool. The first strategy served as a baseline, executing NACD on the complete test suite of 35 tests. The second strategy employed NodeRock,

where NACD was executed only on the subset of tests prioritized by our PU classifier model, chosen for offering the best recall-precision trade-off among all evaluated classifiers (Section 4.1).

Figure 4 compares the total NACD execution time between the two strategies. The filtered suite, comprising 24 tests selected by NodeRock, took a median value of 17,226 s (including instrumentation overhead), compared to 18,331 s for the complete suite of 35 tests. Therefore, the use of NodeRock represents approximately a 6.02 % reduction in runtime¹³. This saving, despite a 31 % reduction in number of tests, suggests that the ML model’s selection included the most complex and long-running tests, which may contain more asynchronous operations and thus dominate the execution time of the test suite.

Importantly, detection capability remained unharmed, with the filtered suite finding a similar number of failures in 100 executions compared to the complete suite (11.5 vs 10.7 on average, respectively), indicating that the tests most relevant to event race detection were preserved by the prioritization model.

Answer to RQ3: Based on the analyzed project, the results suggest that NodeRock is technically feasible with a trade-off. By reducing the test suite by 31 %, it achieved a 6.02 % reduction in total compute time while preserving detection capability, showing potential for saving resources in intensive re-execution scenarios.

¹³Computed as the difference between the median execution times of the complete and filtered suites.

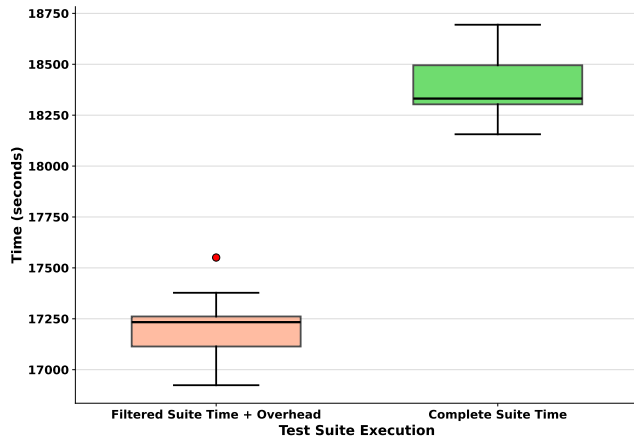


Figure 4: Test Suite Execution Time Comparison (100 runs, repeated 15 times). The Y-axis is truncated to the observed range of values to make the difference between the two strategies more visible.

5 Threats to Validity

The main threat to external validity concerns dataset composition. Although projects were drawn from established academic benchmarks [20, 21, 45], they may not fully represent the diversity of Node.js applications. Therefore, model performance may vary when applied to projects with distinct architectural patterns or domains. Furthermore, the current dependency on Node.js 14 and the Mocha/Jest frameworks restricts immediate applicability to projects outside this ecosystem and leaves out more recent alternatives, such as Vitest and the native Node.js test runner.

Regarding statistical validity, our study is constrained by the relatively small number of positive samples (32 confirmed event races). While significant for this domain, this sample size is modest by ML standards and may limit the robustness of the trained classifier. Part of this positive class comes from isolated test scripts written to expose a specific race, which may not share the execution profile of tests in ordinary suites.

For construct validity, the 15 selected metrics represent a subset of potential characteristics of asynchronous test behavior, and other dimensions not used here may also correlate with event race proneness. Finally, dynamic instrumentation introduces execution overhead. Although data collection is performed in two isolated runs to mitigate measurement interference, we cannot rule out the possibility that the instrumentation overhead may tamper with the timing conditions that trigger race conditions. In particular, the Promise instrumentation writes each settlement event to a log file from the resolution and rejection callbacks, and this I/O adds latency to that execution path. We did not systematically measure whether this extra delay is large enough to alter the timing or outcome of the instrumented tests.

6 Related Work

The quest for reliability in asynchronous environments has led to diverse lines of research. To situate NodeRock within other works,

this section examines four research directions: empirical foundations of concurrency issues in JavaScript, established techniques for automated event race detection, traditional strategies for test suite reduction and prioritization, and the use of machine learning applications in software testing optimization.

Research by Petrov et al. [33] identified that nondeterministic event orderings in concurrent web environments are a major source of reliability issues. Moving this context to the server-side, Wang et al. [42] conducted an extensive study confirming that Node.js applications remain vulnerable to similar patterns, specifically highlighting that atomicity and order violations are prevalent despite the single-threaded nature of the environment. Complementing these empirical findings, Davis et al. [15] introduced Node.fz, demonstrating how controlled scheduling perturbations (fuzzing) can expose race conditions, highlighting the necessity for specialized testing tools in the Node.js ecosystem.

To address the negative impact of event races, the literature diverged into two main automated detection approaches. The first employs *happens-before* graphs to model event dependencies, exemplified by tools like NodeRacer [21], NRace [9], and NodeRT [45]. Although NodeRT improves efficiency by employing an Asynchronous Call Tree, these methods still incur significant costs in modeling execution. The second stream focuses on delay injection, such as NACD [20], which avoids complex modeling in favor of injecting random delays to expose races. However, this stochastic nature often requires a prohibitive number of re-executions to trigger a failure. Common to all these tools is the high computational cost, making exhaustive adoption on large test suites unfeasible. Different from NodeRacer, NRace, NodeRT and NACD, which aim to detect event races within a given test, NodeRock targets a different research problem: selecting and prioritizing which tests are worth submitting to those tools. In practice, NodeRock would be executed first, producing a prioritized subset of tests prone to event races, over which developers can then apply a detection tool.

To mitigate testing costs, the software engineering field established TCP and TSR. TCP aims to reorder tests to maximize fault detection rates [18, 34], while TSR seeks to select a minimal representative subset of tests [14, 24]. While effective for regression testing, both strategies rely on heuristics grounded in code coverage, change impact, or historical failure data. These criteria do not have direct sensitivity to the nondeterministic nature of event races. A test with no prior failures and low code coverage may still be capable of exposing an event race, and standard TCP/TSR heuristics would not identify it as a priority, potentially discarding it.

ML has also evolved to optimize testing. Unsupervised approaches, such as those by Chen et al. [10], have used clustering based on execution profiles to group similar tests. On the supervised front, researchers like Busjaeger and Xie [6] automated prioritization using SVM models trained on coverage and keyword metrics. More recently, Reinforcement Learning has been explored to adapt to the continuous changes in Continuous Integration (CI) environments [1]. However, these ML applications typically rely on traditional software engineering features (e.g., complexity, lines of code) rather than dynamic metrics specific to asynchronous runtime behavior. Camara et al. [7] replicated a study that predicts test flakiness from the vocabulary of code identifiers using machine

learning and found that, while classifiers perform well within the training projects, their recall drops when applied to other projects.

The literature presents a gap between effective but computationally expensive race detection tools [9, 20, 21, 45] and efficient test optimization techniques that are not designed for event race prioritization [1, 14]. NodeRock aims to bridge this gap. Unlike existing TCP/TSR techniques, we employ dynamic analysis and ML specialized for the asynchronous domain of JavaScript and Node.js. By identifying tests prone to event races based on their execution profile, our approach does not replace detection tools like NodeRacer, NRace, NodeRT and NACD, but rather complements them, directing debugging efforts to where they are most needed and making concurrency testing practical at scale.

7 Conclusion and Future Work

We have presented NodeRock, an approach designed to address the challenge of detecting event races in Node.js applications. This challenge escalates with software complexity, and existing solutions often impose excessive computational costs. To mitigate this, NodeRock uses dynamic analysis and ML not to deterministically detect bugs, but to select and prioritize tests prone to event races. By focusing on the analysis of asynchronous execution profiles, our approach optimizes the debugging effort, making the identification of concurrency issues more practical for large-scale projects.

Our empirical evaluation across 24 open-source projects provides evidence in support of three findings. First, the asynchronous execution profile of a test quantified through callback use, asynchronous function duration, and Promise lifecycle metrics constitutes a meaningful indicator of event race proneness. Second, among the classifiers evaluated, the PU classifier offered the most suitable trade-off for this problem, achieving 84.38 % recall while maintaining 75 % accuracy and 50 % precision. Third, in a study restricted to a single project, NodeRock reduced total execution time by 6.02 % in an intensive re-execution scenario while preserving detection capability.

These results should be interpreted with appropriate caution. The 84.38 % recall implies that a small but nonzero fraction of tests prone to event races will be incorrectly deprioritized. NodeRock should therefore be understood as a risk optimization strategy. It allows teams to concentrate the majority of their debugging resources on high-risk tests, but does not eliminate the need for periodic full-suite analyses. Furthermore, the current dataset of 32 confirmed event races, while representative of the academic benchmark literature, constrains the statistical robustness of the trained model and limits generalization.

For future investigation, the feature set could be extended with other static analysis metrics, such as cyclomatic complexity, code coverage and runtime data including garbage collection pause times, which may alter Event Loop context and correlate with race manifestation. The dataset should be expanded with a larger and more diverse corpus of confirmed event races, ideally collected from real-world projects outside the existing benchmarks, which would also enable the exploration of more expressive learning architectures, such as deep neural networks, and allow more rigorous hyperparameter optimization. An expanded dataset would also support comparing NodeRock against a Large Language Model (LLM) prompted to

select and prioritize tests prone to event races directly from source code, without the dynamic instrumentation step. A hybrid analysis architecture, in which a lightweight static pre-filter eliminates manifestly synchronous tests before dynamic instrumentation is applied, could reduce the fixed overhead that currently limits NodeRock's utility in high-frequency CI pipelines. Finally, the execution profile selection strategy explored here is not specific to JavaScript event races. The approach could be adapted to other concurrency bugs or to other asynchronous ecosystems, such as Python's `asyncio`¹⁴ or C# `async/await`.

Artifact Availability

To support the reproducibility of our study and facilitate future research, all the artifacts produced during NodeRock development, including the JavaScript instrumentation logic, feature extraction scripts, the Python-based machine learning model, and benchmark logs and data are presented, under the GNU license, in our Zenodo repository: <https://doi.org/10.5281/zenodo.22968741>.

Acknowledgements

P.V.G. Vicente is financially supported by Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001 and São Paulo Research Foundation - FAPESP (grant #2024/05445-5). This work is supported by FAPESP (grant #2023/00577-8) and by the National Council for Scientific and Technological Development - CNPq (grants nr. 444311/2024-6 and 406941/2025-4).

References

- [1] Mojtaba Bagherzadeh, Nafiseh Kahani, and Lionel Briand. 2022. Reinforcement Learning for Test Case Prioritization. *IEEE Transactions on Software Engineering* 48, 8 (2022), 2836–2856. doi:10.1109/TSE.2021.3070549
- [2] Alexander Berndt, Thomas Bach, and Sebastian Baltes. 2024. Do Test and Environmental Complexity Increase Flakiness? An Empirical Study of SAP HANA. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (Barcelona, Spain) (ESEM '24)*. Association for Computing Machinery, New York, NY, USA, 572–581. doi:10.1145/3674805.3695407
- [3] Leo Breiman. 2001. Random forests. *Machine learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324
- [4] Leo Breiman, Jerome Friedman, Charles J Stone, and Richard A Olshen. 1984. *Classification and Regression Trees*. CRC press.
- [5] BrowserStack. 2025. "What is Monkey Patching?". [Online]. Available: <https://www.browserstack.com/guide/monkey-patching>.
- [6] Benjamin Busjaeger and Tao Xie. 2016. Learning for test prioritization: an industrial case study. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (Seattle, WA, USA) (FSE 2016)*. Association for Computing Machinery, New York, NY, USA, 975–980. doi:10.1145/2950290.2983954
- [7] Bruno Henrique Pachulski Camara, Marco Aurélio Graciotto Silva, Andre T. Endo, and Silvia Regina Vergilio. 2021. What is the Vocabulary of Flaky Tests? An Extended Replication. In *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. 444–454. doi:10.1109/ICPC52881.2021.00052
- [8] Ricardo J. G. B. Campello, Davoud Moulavi, and Joerg Sander. 2013. Density-Based Clustering Based on Hierarchical Density Estimates. In *Advances in Knowledge Discovery and Data Mining*, Jian Pei, Vincent S. Tseng, Longbing Cao, Hiroshi Motoda, and Guandong Xu (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 160–172.
- [9] Xiaoning Chang, Wensheng Dou, Jun Wei, Tao Huang, Jinhui Xie, Yuetang Deng, Jianbo Yang, and Jiaheng Yang. 2021. Race Detection for Event-Driven Node.js Applications. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 480–491. doi:10.1109/ASE51524.2021.9678814
- [10] Songyu Chen, Zhenyu Chen, Zhihong Zhao, Baowen Xu, and Yang Feng. 2011. Using semi-supervised clustering to improve regression test selection techniques.

¹⁴<https://docs.python.org/3/library/asyncio.html>

- In 2011 Fourth IEEE International Conference on Software Testing, Verification and Validation. 1–10. doi:10.1109/ICST.2011.38
- [11] Nat Chrzanowska. 2024. “12 Top Applications Written in Node.js - Examples from Big Companies”. [Online]. Available: <https://www.netguru.com/blog/top-companies-used-nodejs-production>.
- [12] Corinna Cortes and Vladimir Vapnik. 1995. Support-Vector Networks. *Mach. Learn.* 20, 3 (Sept. 1995), 273–297. doi:10.1023/A:1022627411411
- [13] T. Cover and P. Hart. 1967. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory* 13, 1 (1967), 21–27. doi:10.1109/TIT.1967.1053964
- [14] Emilio Cruciani, Breno Miranda, Roberto Verdecchia, and Antonia Bertolino. 2019. Scalable Approaches for Test Suite Reduction. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 419–429. doi:10.1109/ICSE.2019.00055
- [15] James Davis, Arun Thekumparampil, and Dongyoon Lee. 2017. Node.fz: Fuzzing the Server-Side Event-Driven Architecture. In *Proceedings of the Twelfth European Conference on Computer Systems (Belgrade, Serbia) (EuroSys '17)*. Association for Computing Machinery, New York, NY, USA, 145–160. doi:10.1145/3064176.3064188
- [16] Richard O. Duda, Peter E. Hart, and David G. Stork. 2000. *Pattern Classification (2nd Edition)*. Wiley-Interscience, USA.
- [17] Ecma International. 2025. “ECMAScript® 2025 Language Specification”. [Online]. Available: <https://262.ecma-international.org/#sec-promise-objects>.
- [18] S. Elbaum, A.G. Malishevsky, and G. Rothermel. 2002. Test case prioritization: a family of empirical studies. *IEEE Transactions on Software Engineering* 28, 2 (2002), 159–182. doi:10.1109/32.988497
- [19] Charles Elkan and Keith Noto. 2008. Learning classifiers from only positive and unlabeled data. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (Las Vegas, Nevada, USA) (KDD '08)*. Association for Computing Machinery, New York, NY, USA, 213–220. doi:10.1145/1401890.1401920
- [20] André Takeshi Endo and Anders Møller. 2025. Event Race Detection for Node.js Using Delay Injections. In *39th European Conference on Object-Oriented Programming (ECOOP 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 333)*, Jonathan Aldrich and Alexandra Silva (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:28. doi:10.4230/LIPIcs.ECOOP.2025.9
- [21] André Takeshi Endo and Anders Møller. 2020. NodeRacer: Event Race Detection for Node.js Applications. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. 120–130. doi:10.1109/ICST46399.2020.00022
- [22] GitHub Staff. 2025. “GitHub Octoverse 2025”. [Online]. Available: <https://octoverse.github.com/>.
- [23] Isabelle Guyon and André Elisseeff. 2003. An introduction to variable and feature selection. *J. Mach. Learn. Res.* 3, null (March 2003), 1157–1182.
- [24] M. Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. 1993. A methodology for controlling the size of a test suite. *ACM Trans. Softw. Eng. Methodol.* 2, 3 (July 1993), 270–285. doi:10.1145/152388.152391
- [25] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2008. Isolation Forest. In *2008 Eighth IEEE International Conference on Data Mining*. 413–422. doi:10.1109/ICDM.2008.17
- [26] S. Lloyd. 1982. Least squares quantization in PCM. *IEEE Transactions on Information Theory* 28, 2 (1982), 129–137. doi:10.1109/TIT.1982.1056489
- [27] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, (FSE-22)*, Hong Kong, China, November 16 - 22, 2014, Shing-Chi Cheung, Alessandro Orso, and Margaret-Anne D. Storey (Eds.). ACM, 643–653. doi:10.1145/2635868.2635920
- [28] Wes McKinney. 2010. Data Structures for Statistical Computing in Python. *SciPy 2010* (2010). doi:10.25080/Majora-92bf1922-00a
- [29] Mozilla Developer Network. 2025. “Promise”. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise.
- [30] Node.js Contributors. 2025. “The Node.js Event Loop”. [Online]. Available: <https://nodejs.org/learn/asynchronous-work/event-loop-timers-and-nexttick>.
- [31] Rongqi Pan, Mojtaba Bagherzadeh, Taher A. Ghaleb, and Lionel Briand. 2021. Test case selection and prioritization using machine learning: a systematic literature review. *Empirical Software Engineering* 27, 2 (dec 2021), 29. doi:10.1007/s10664-021-10066-6
- [32] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12, 85 (2011), 2825–2830. <http://jmlr.org/papers/v12/pedregosa11a.html>
- [33] Boris Petrov, Martin Vechev, Manu Sridharan, and Julian Dolby. 2012. Race detection for web applications. *SIGPLAN Not.* 47, 6 (June 2012), 251–262. doi:10.1145/2345156.2254095
- [34] G. Rothermel, R.H. Untch, Chengyun Chu, and M.J. Harrold. 2001. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering* 27, 10 (2001), 929–948. doi:10.1109/32.962562
- [35] Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. 2001. Estimating the Support of a High-Dimensional Distribution. *Neural Computation* 13, 7 (2001), 1443–1471. doi:10.1162/089976601750264965
- [36] Shweta Singhal, Nishtha Jatana, Bharti Suri, Sanjay Misra, and Luis Fernandez-Sanz. 2021. Systematic Literature Review on Test Case Selection and Prioritization: A Tertiary Study. *Applied Sciences* 11, 24 (2021). doi:10.3390/app112412121
- [37] Stack Exchange, Inc. 2025. “Stack Overflow Developer Survey 2025”. [Online]. Available: <https://survey.stackoverflow.co/2025/technology>.
- [38] Hervé Stoppiglia, Gérard Dreyfus, Rémi Dubois, and Yacine Oussar. 2003. Ranking a random feature for variable and feature selection. *J. Mach. Learn. Res.* 3, null (March 2003), 1399–1414.
- [39] Haiyang Sun, Daniele Bonetta, Christian Humer, and Walter Binder. 2018. Efficient Dynamic Analysis for Node.js. In *Proceedings of the 27th International Conference on Compiler Construction (Vienna, Austria) (CC 2018)*. ACM, New York, NY, USA, 196–206. doi:10.1145/3178372.3179527
- [40] Stefan Tilkov and Steve Vinoski. 2010. Node.js: Using JavaScript to Build High-Performance Network Programs. *IEEE Internet Computing* 14, 6 (2010), 80–83. doi:10.1109/MIC.2010.145
- [41] TIOBE Software BV. 2026. “TIOBE Index for June 2026”. [Online]. Available: <https://www.tiobe.com/tiobe-index/>.
- [42] Jie Wang, Wensheng Dou, Yu Gao, Chushu Gao, Feng Qin, Kang Yin, and Jun Wei. 2017. A comprehensive study on real world concurrency bugs in Node.js. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 520–531. doi:10.1109/ASE.2017.8115663
- [43] S. Yoo and M. Harman. 2012. Regression testing minimization, selection and prioritization: a survey. *Softw. Test. Verif. Reliab.* 22, 2 (March 2012), 67–120. doi:10.1002/9781119896838.ch7
- [44] Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. 2003. Learning with local and global consistency. In *Proceedings of the 17th International Conference on Neural Information Processing Systems (Whistler, British Columbia, Canada) (NIPS'03)*. MIT Press, Cambridge, MA, USA, 321–328.
- [45] Jingyao Zhou, Lei Xu, Gongzheng Lu, Weifeng Zhang, and Xiangyu Zhang. 2023. NodeRT: Detecting Races in Node.js Applications Practically. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (Seattle, WA, USA) (ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1332–1344. doi:10.1145/3597926.3598139